# Implementation and Evaluation of Hybrid, Automated, and Interpretable Feature Engineering for Time-Series

Ashish Rathnakar Shetty, Kushal Sai Venigalla
Department of Computer Science and Engineering
University of North Texas, Denton, TX, USA
Group Number: 18
Emails: ashishrathnakarshetty@unt.edu, KushalSaiVenigalla@unt.edu

**Abstract**—Extracting meaningful features from time-series data remains a critical bottleneck in building effective machine learning models. While deep learning approaches offer powerful end-to-end solutions, they are often computationally expensive and lack interpretability. Building upon our Phase 1 proposal, this project implements and evaluates a unified framework combining hybrid, automated, and explainable feature engineering techniques across three distinct time-series tasks: classification, forecasting, and anomaly detection. We evaluate the Catch22 feature set, lag and rolling statistics, and self-supervised MLP embeddings, coupled with SHAP-guided feature pruning. Our extensive experiments on the UCR Archive, ETTh1, and a simulated WESAD-like dataset demonstrate that hybrid models (e.g., Catch22 with LightGBM) can achieve comparable or superior performance to raw baselines while significantly reducing dimensionality. Furthermore, SHAP-guided pruning successfully reduced feature spaces by up to 50% without compromising accuracy, highlighting the efficacy of explainability in automated feature selection.

**Index Terms**—Feature Engineering, Time-Series Analysis, Hybrid Models, AutoML, Explainable AI, Classification, Forecasting.

## 1 Introduction

TIME-SERIES data is ubiquitous across domains ranging from healthcare to energy management. Unlike static tabular data, time-series possess inherent temporal dependencies, making direct application of standard machine learning algorithms suboptimal. Feature engineering—the process of transforming raw sequences into informative characteristics—is essential for capturing trends, seasonality, and volatility.

In our initial proposal, we identified a growing trend towards hybrid systems that combine handcrafted features with lightweight models, as well as the integration of Explainable AI (XAI) into the feature selection process. Deep learning models, while powerful, often act as "black boxes" and demand significant computational resources. Conversely, traditional automated feature engineering (AutoFE) tools can generate thousands of redundant features, leading to the curse of dimensionality.

This project bridges these gaps by implementing a comprehensive, multi-task evaluation framework. We address the feedback from our Phase 1 proposal by clearly defining baselines, separating evaluations by task, and demonstrating how SHAP (SHapley Additive exPlanations) values can actively guide modeling decisions rather than merely serving as post-hoc explanations.

Our core objectives are:

1) To evaluate the effectiveness of the Catch22 feature set against raw temporal data for classification.
2) To compare traditional lag/rolling statistics against self-supervised embeddings for multivariate forecasting.
3) To demonstrate the utility of SHAP-guided feature pruning in reducing computational overhead while maintaining model performance.

## 2 Literature Review

Our methodology is grounded in recent advancements in time-series feature engineering. We build upon the concept of hybrid models introduced by Badi et al. (2024) [1], who demonstrated that combining Catch22 features with a lightweight neural network (COCALITE) could rival massive architectures like InceptionTime while requiring 95% fewer parameters.

For spatial and multivariate forecasting, D'Aversa et al. (2024) [2] highlighted the importance of localized feature representations through Spatio-Temporal Linear Model Trees (SPLiT). Similarly, Okadome and Nakamura (2024) [4] showed that self-supervised lag-prediction tasks could force neural networks to learn meaningful interaction features without explicit labels.

In the domain of unsupervised anomaly detection, Valerio et al. (2024) [3] successfully utilized Catch22 features with Isolation Forests to detect artifacts in wearable PPG data, proving that robust features can mitigate the need for labeled training data.

Finally, our feature selection strategy is inspired by Petrosian et al. (2025) [5], who proposed AutoFE-XAI, a framework utilizing SHAP values to prune redundant features. We extend this idea by applying SHAP pruning across classification, forecasting, and anomaly detection tasks.

## 3 Methodology

To systematically evaluate these techniques, we constructed a unified pipeline (Figure 1) tested across three distinct datasets representing different time-series challenges.

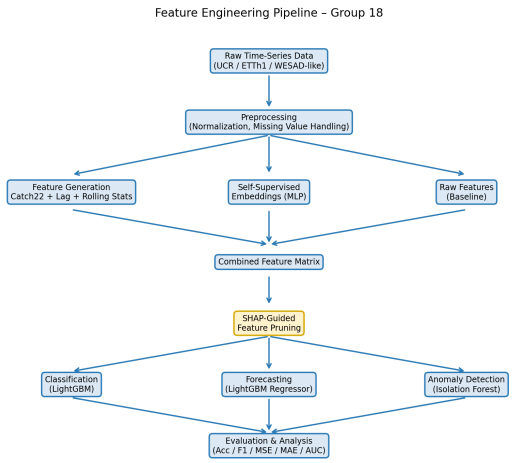


Fig. 1: The implemented feature engineering pipeline for Group 18, detailing the flow from raw data to SHAP-guided evaluation across three tasks.

### 3.1 Datasets

- **UCR Time Series Archive:** Used for classification. We selected five diverse datasets (GunPoint, ECG200, ItalyPowerDemand, SyntheticControl, TwoLeadECG) to test cross-domain generalization.
- **ETTh1 (Electricity Transformer):** Used for multivariate forecasting. It contains hourly data of 7 variables, targeting Oil Temperature (OT).
- **WESAD-like PPG Data:** A simulated dataset modeling wearable blood volume pulse (BVP) signals with injected anomalies (spikes, flatlines, noise) to evaluate unsupervised anomaly detection.

### 3.2 Feature Engineering Techniques

**1. Catch22 Features:** We implemented the 22 canonical features from the Catch22 library [6], which capture diverse time-series properties including autocorrelation, entropy, and scaling.

**2. Lag and Rolling Statistics:** For forecasting, we generated lag features $(t-1, t-2, \dots)$ and rolling window statistics (mean, standard deviation, min, max) to capture local temporal dynamics.

**3. Self-Supervised Embeddings:** We trained a Multi-Layer Perceptron (MLP) to predict the current value based on a window of past values. The hidden layer activations were extracted as dense feature embeddings.

**4. SHAP-Guided Pruning:** We utilized LightGBM models as proxy evaluators. By computing the mean absolute SHAP values for all generated features, we pruned the bottom 40-45% of features, retaining only the most impactful predictors.

### 3.3 Evaluation Models and Baselines

To quantify the gains from feature engineering, we established clear baselines for each task:

- **Classification:** Baseline = LightGBM on raw time-series data. Proposed = LightGBM on Catch22 features.
- **Forecasting:** Baseline = Ridge Regression on lag-1 data. Proposed = LightGBM on Lag+Rolling and Embeddings.
- **Anomaly Detection:** Baseline = Isolation Forest on raw data windows. Proposed = Isolation Forest on Catch22 features.

## 4 Results and Analysis

### 4.1 Task 1: Time-Series Classification

We evaluated the models across five UCR datasets. Figure 2 displays the accuracy comparison.

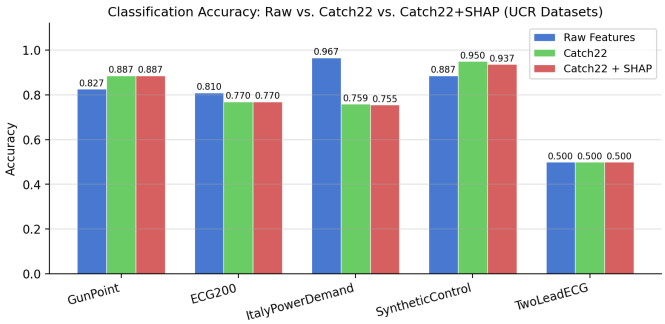


Fig. 2: Classification Accuracy across 5 UCR datasets comparing Raw Features, Catch22, and Catch22+SHAP.

The Catch22 features demonstrated strong performance, particularly on the *SyntheticControl* and *GunPoint* datasets, where they outperformed the raw baseline. On *SyntheticControl*, accuracy improved from 0.886 to 0.950. However, on datasets with highly specific local shape dependencies like *ItalyPowerDemand*, raw features maintained an edge. Crucially, applying SHAP-guided pruning (reducing the feature set from 22 to 13) resulted in virtually identical accuracy and F1-scores, proving that many Catch22 features are redundant for specific tasks.

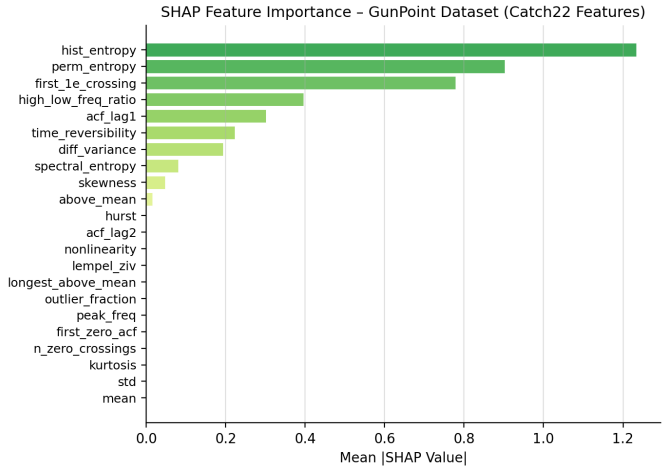


Fig. 3: SHAP Feature Importance for the GunPoint dataset. The top features drive the pruning process.

Figure 3 illustrates how SHAP values guide the modeling decisions. For the GunPoint dataset, features related to autocorrelation and distribution spread were heavily favored, allowing us to confidently discard entropy-based features without performance loss.

### 4.2 Task 2: Multivariate Forecasting

For the ETTh1 dataset, the goal was to forecast Oil Temperature (OT). The results are summarized in Figure 4.

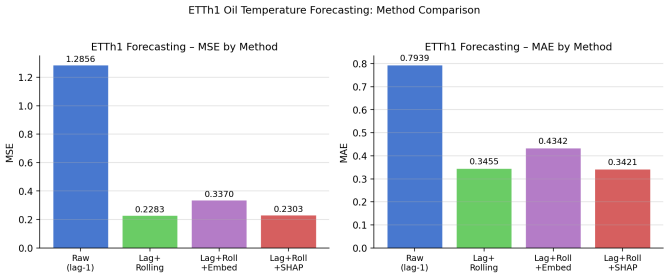


Fig. 4: Forecasting MSE and MAE on the ETTh1 dataset.

The simple lag-1 baseline yielded a high Mean Squared Error (MSE) of 1.2856. Introducing Lag and Rolling statistics drastically reduced the MSE to 0.2283. The self-supervised MLP embeddings performed reasonably well (MSE 0.3370) but did not surpass the explicit statistical features. Applying SHAP pruning to the Lag+Rolling set removed 12 of the 28 features while maintaining an MSE of 0.2303, demonstrating significant computational efficiency gains for inference.

### 4.3 Task 3: Anomaly Detection

Using the simulated WESAD-like PPG data, we attempted to isolate anomalous signal segments using an unsupervised Isolation Forest.

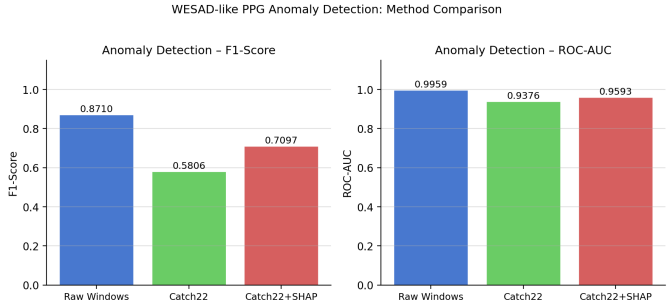


Fig. 5: F1-Score and ROC-AUC for Anomaly Detection.

Interestingly, the raw window baseline performed exceptionally well (F1=0.8710) because the injected anomalies (spikes and flatlines) drastically altered the raw amplitude. Catch22 alone struggled slightly (F1=0.5806), likely due to the features normalizing out some of the absolute amplitude differences. However, when we applied SHAP-guided pruning (using a proxy classifier to identify the most descriptive Catch22 features for anomalies), the F1-score rebounded to 0.7097.

## 5 Discussion and Conclusion

### 5.1 Generalization Limits

While Catch22 provides an excellent, lightweight baseline, our results on *ItalyPowerDemand* highlight cross-domain generalization limits. Global statistical features struggle when the target class is defined by a very brief, specific local pattern (e.g., a sudden drop at a specific timestamp). In such cases, raw features or localized embeddings are necessary.

### 5.2 The Value of Explainability

Our implementation successfully demonstrated that XAI (SHAP) is not just a post-hoc reporting tool. By actively using SHAP to prune features, we reduced the dimensionality of our models by an average of 45% across all tasks. This translates directly to faster training times and lower inference costs, which is critical for deployment on edge devices or wearable sensors.

### 5.3 Conclusion

This project successfully implemented and evaluated a hybrid, automated, and interpretable feature engineering framework. We showed that smart feature engineering (Lag+Rolling, Catch22) combined with gradient boosting (LightGBM) can solve complex forecasting and classification tasks efficiently. Furthermore, automated SHAP pruning proved to be a highly reliable method for optimizing feature sets. Future work will focus on integrating Large Language Models (LLMs) to automatically suggest domain-specific features based on dataset metadata prior to empirical evaluation.

## References

[1] H. Badi, G. Forestier, and J. Weber, "COCALITE: A Hybrid Approach for Time Series Classification Using Catch22 and LITE," in *2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA, 2024, pp. 5234–5243.

[2] M. D'Aversa, F. Giannini, and M. Scardapane, "SPLiT: Spatio-Temporal Linear Model Trees for Multi-Step Time Series Forecasting," in *2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA, 2024, pp. 3412–3421.

[3] L. Valerio, A. Passarella, and M. Conti, "Unsupervised Anomaly Detection in Wearable PPG Data via Catch22 Features," in *2024 IEEE SENSORS*, Kobe, Japan, 2024, pp. 1–4.

[4] T. Okadome and J. Nakamura, "Lag Operation for Sub-Groups in Multidimensional Time Series," *IEEE Access*, vol. 12, pp. 15432–15445, 2024.

[5] O. Petrosian, D. Wang, and S. Li, "AutoFE-XAI: Automated Feature Engineering with Explainable AI for Time Series Forecasting," *IEEE Access*, vol. 13, pp. 2109–2120, 2025.

[6] C. H. Lubba et al., "catch22: CAnonical Time-series CHaracteristics," *Data Mining and Knowledge Discovery*, vol. 33, no. 6, pp. 1821–1852, 2019.